

Traiter une série d'éléments avec un itérateur

Les itérateurs vous permettent d'effectuer une tâche sur une séquence d'éléments à tour de rôle. Un *itérateur* est responsable de la logique d'itération sur chaque élément et de déterminer lorsque la séquence est terminée. Lorsque nous utilisons des itérateurs, nous n'avons pas besoin de ré-implémenter cette logique nous-mêmes.

En Rust, un itérateur est *une évaluation paresseuse*, ce qui signifie qu'il n'a aucun effet jusqu'à ce que nous appelions des méthodes qui consomment l'itérateur pour l'utiliser. Par exemple, le code dans l'encart 13-13 crée un itérateur sur les éléments du vecteur `v1` en appelant la méthode `iter` définie sur `Vec<T>`. Ce code en lui-même ne fait rien d'utile.

```
let v1 = vec![1, 2, 3];  
  
let v1_iter = v1.iter();
```

Encart 13-13 : création d'un itérateur

Une fois que nous avons créé un itérateur, nous pouvons l'utiliser de diverses manières. Dans l'encart 3-4 du chapitre 3, nous avons utilisé des itérateurs avec des boucles `for` pour exécuter du code sur chaque élément, bien que nous ayons laissé de côté ce que l'appel à `iter` faisait jusqu'à présent.

L'exemple dans l'encart 13-14 sépare la création de l'itérateur de son utilisation dans la boucle `for`. L'itérateur est stocké dans la variable `v1_iter`, et aucune itération n'a lieu à ce moment-là. Lorsque la boucle `for` est appelée en utilisant l'itérateur `v1_iter`, chaque élément de l'itérateur est utilisé à chaque itération de la boucle, qui affiche chaque valeur.

```
let v1 = vec![1, 2, 3];  
  
let v1_iter = v1.iter();  
  
for val in v1_iter {  
    println!("On a : {}", val);  
}
```

Encart 13-14 : utilisation d'un itérateur dans une boucle `for`

Dans les langages qui n'ont pas d'itérateurs fournis par leur bibliothèque standard, nous écririons probablement cette même fonctionnalité en démarrant une variable à l'indice 0,

en utilisant cette variable comme indice sur le vecteur afin d'obtenir une valeur puis en incrémentant la valeur de cette variable dans une boucle jusqu'à ce qu'elle atteigne le nombre total d'éléments dans le vecteur.

Les itérateurs s'occupent de toute cette logique pour nous, réduisant le code redondant dans lequel nous pourrions potentiellement faire des erreurs. Les itérateurs nous donnent plus de flexibilité pour utiliser la même logique avec de nombreux types de séquences différentes, et pas seulement avec des structures de données avec lesquelles nous pouvons utiliser des indices, telles que les vecteurs. Voyons comment les itérateurs font cela.

Le trait `Iterator` et la méthode `next`

Tous les itérateurs implémentent un trait appelé `Iterator` qui est défini dans la bibliothèque standard. La définition du trait ressemble à ceci :

```
pub trait Iterator {  
    type Item;  
  
    fn next(&mut self) -> Option<Self::Item>;  
  
    // les méthodes avec des implémentations par défaut ont été exclues  
}
```

Remarquez que cette définition utilise une nouvelle syntaxe : `type Item` et `Self::Item`, qui définissent un *type associé* à ce trait. Nous verrons ce que sont les types associés au chapitre 19. Pour l'instant, tout ce que vous devez savoir est que ce code dit que l'implémentation du trait `Iterator` nécessite que vous définissiez aussi un type `Item`, et ce type `Item` est utilisé dans le type de retour de la méthode `next`. En d'autres termes, le type `Item` sera le type retourné par l'itérateur.

Le trait `Iterator` exige la définition d'une seule méthode par les développeurs : la méthode `next`, qui retourne un élément de l'itérateur à la fois intégré dans un `Some`, et lorsque l'itération est terminée, il retourne `None`.

On peut appeler la méthode `next` directement sur les itérateurs ; l'encart 13-15 montre quelles valeurs sont retournées par des appels répétés à `next` sur l'itérateur créé à partir du vecteur.

Fichier : `src/lib.rs`

```
#[test]
fn demo_iterateur() {
    let v1 = vec![1, 2, 3];

    let mut v1_iter = v1.iter();

    assert_eq!(v1_iter.next(), Some(&1));
    assert_eq!(v1_iter.next(), Some(&2));
    assert_eq!(v1_iter.next(), Some(&3));
    assert_eq!(v1_iter.next(), None);
}
```

Encart 13-15 : appel de la méthode `next` sur un itérateur

Remarquez que nous avons eu besoin de rendre mutable `v1_iter` : appeler la méthode `next` sur un iterator change son état interne qui garde en mémoire l'endroit où il en est dans la séquence. En d'autres termes, ce code *consomme*, ou utilise, l'itérateur. Chaque appel à `next` consomme un élément de l'itérateur. Nous n'avions pas eu besoin de rendre mutable `v1_iter` lorsque nous avons utilisé une boucle `for` parce que la boucle avait pris possession de `v1_iter` et l'avait rendu mutable en coulisses.

Notez également que les valeurs que nous obtenons des appels à `next` sont des références immuables aux valeurs dans le vecteur. La méthode `iter` produit un itérateur pour des références immuables. Si nous voulons créer un itérateur qui prend possession de `v1` et retourne les valeurs possédées, nous pouvons appeler `into_iter` au lieu de `iter`. De même, si nous voulons itérer sur des références mutables, nous pouvons appeler `iter_mut` au lieu de `iter`.

Les méthodes qui consomment un itérateur

Le trait `Iterator` a un certain nombre de méthodes différentes avec des implémentations par défaut que nous fournit la bibliothèque standard ; vous pouvez découvrir ces méthodes en regardant dans la documentation de l'API de la bibliothèque standard pour le trait `Iterator`. Certaines de ces méthodes appellent la méthode `next` dans leur définition, c'est pourquoi nous devons toujours implémenter la méthode `next` lors de l'implémentation du trait `Iterator`.

Les méthodes qui appellent `next` sont appelées des *adaptateurs de consommation*, parce que les appeler consomme l'itérateur. Un exemple est la méthode `sum`, qui prend possession de l'itérateur et itère sur ses éléments en appelant plusieurs fois `next`,

consommant ainsi l'itérateur. A chaque étape de l'itération, il ajoute chaque élément à un total en cours et retourne le total une fois l'itération terminée. L'encart 13-16 a un test illustrant une utilisation de la méthode `sum` :

Fichier : `src/lib.rs`

```
#[test]
fn iterator_sum() {
    let v1 = vec![1, 2, 3];

    let v1_iter = v1.iter();

    let total: i32 = v1_iter.sum();

    assert_eq!(total, 6);
}
```

Encart 13-16 : appel de la méthode `sum` pour obtenir la somme de tous les éléments présents dans l'itérateur

Nous ne sommes pas autorisés à utiliser `v1_iter` après l'appel à `sum` car `sum` a pris possession de l'itérateur avec lequel nous l'appelons.

Méthodes qui produisent d'autres itérateurs

D'autres méthodes définies sur le trait `Iterator`, connues sous le nom *d'adaptateurs d'itération*, nous permettent de transformer un itérateur en un type d'itérateur différent. Nous pouvons enchaîner plusieurs appels à des adaptateurs d'itération pour effectuer des actions complexes de manière compréhensible. Mais comme les itérateurs sont *des évaluations paresseuses*, nous devons faire appel à l'une des méthodes d'adaptation de consommation pour obtenir les résultats des appels aux adaptateurs d'itération.

L'encart 13-17 montre un exemple d'appel à la méthode d'adaptation d'itération `map`, qui prend en paramètre une fermeture qui va s'exécuter sur chaque élément pour produire un nouvel itérateur. La fermeture crée ici un nouvel itérateur dans lequel chaque élément du vecteur a été incrémenté de 1. Cependant, ce code déclenche un avertissement :

Fichier : `src/main.rs`

```
let v1: Vec<i32> = vec![1, 2, 3];

v1.iter().map(|x| x + 1);
```

Encart 13-17 : appel de l'adaptateur d'itération `map` pour créer un nouvel itérateur

Voici l'avertissement que nous obtenons :

```
$ cargo run
  Compiling iterators v0.1.0 (file:///projects/iterators)
warning: unused `Map` that must be used
--> src/main.rs:4:5
4 |         v1.iter().map(|x| x + 1);
  |         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
= note: `#[warn(unused_must_use)]` on by default
= note: iterators are lazy and do nothing unless consumed

warning: `iterators` (bin "iterators") generated 1 warning
  Finished dev [unoptimized + debuginfo] target(s) in 0.47s
  Running `target/debug/iterators`
```

Le code dans l'encart 13-17 ne fait rien ; la fermeture que nous avons renseignée n'est jamais exécutée. L'avertissement nous rappelle pourquoi : les adaptateurs d'itération sont des *évaluations paresseuses*, c'est pourquoi nous devons consommer l'itérateur ici.

Pour corriger ceci et consommer l'itérateur, nous utiliserons la méthode `collect`, que vous avez utilisé avec `env::args` dans l'encart 12-1 du chapitre 12. Cette méthode consomme l'itérateur et collecte les valeurs résultantes dans un type de collection de données.

Dans l'encart 13-18, nous recueillons les résultats de l'itération sur l'itérateur qui sont retournés par l'appel à `map` sur un vecteur. Ce vecteur finira par contenir chaque élément du vecteur original incrémenté de 1.

Fichier : src/main.rs

```
let v1: Vec<i32> = vec![1, 2, 3];

let v2: Vec<_> = v1.iter().map(|x| x + 1).collect();

assert_eq!(v2, vec![2, 3, 4]);
```

Encart 13-18 : appel de la méthode `map` pour créer un nouvel itérateur, puis appel de la méthode `collect` pour consommer le nouvel itérateur afin de créer un vecteur

Comme `map` prend en paramètre une fermeture, nous pouvons renseigner n'importe quelle opération que nous souhaitons exécuter sur chaque élément. C'est un bon

exemple de la façon dont les fermetures nous permettent de personnaliser certains comportements tout en réutilisant le comportement d'itération fourni par le trait `Iterator`.

Utilisation de fermetures capturant leur environnement

Maintenant que nous avons présenté les itérateurs, nous pouvons illustrer une utilisation commune des fermetures qui capturent leur environnement en utilisant l'adaptateur d'itération `filter`. La méthode `filter` appelée sur un itérateur prend en paramètre une fermeture qui s'exécute sur chaque élément de l'itérateur et retourne un booléen pour chacun. Si la fermeture retourne `true`, la valeur sera incluse dans l'itérateur produit par `filter`. Si la fermeture retourne `false`, la valeur ne sera pas incluse dans l'itérateur résultant.

Dans l'encart 13-19, nous utilisons `filter` avec une fermeture qui capture la variable `pointure_chaussure` de son environnement pour itérer sur une collection d'instances de la structure `Chaussure`. Il ne retournera que les chaussures avec la pointure demandée.

Fichier : src/lib.rs

```
#[derive(PartialEq, Debug)]
struct Chaussure {
    pointure: u32,
    style: String,
}

fn chaussures_a_la_pointure(chaussures: Vec<Chaussure>, pointure_chaussure:
u32) -> Vec<Chaussure> {
    chaussures.into_iter()
        .filter(|s| s.pointure == pointure_chaussure)
        .collect()
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn filtres_par_pointure() {
        let chaussures = vec![
            Chaussure {
                pointure: 10,
                style: String::from("baskets"),
            },
        ],
```

```

        Chaussure {
            pointure: 13,
            style: String::from("sandale"),
        },
        Chaussure {
            pointure: 10,
            style: String::from("bottes"),
        },
    ];

    let a_ma_pointure = chaussures_a_la_pointure(chaussures, 10);

    assert_eq!(
        a_ma_pointure,
        vec![
            Chaussure {
                pointure: 10,
                style: String::from("baskets")
            },
            Chaussure {
                pointure: 10,
                style: String::from("bottes")
            },
        ]
    );
}

```

Encart 13-19 : utilisation de la méthode `filter` avec une fermeture capturant `pointure_chaussure`

La fonction `chaussures_a_la_pointure` prend possession d'un vecteur de chaussures et d'une pointure comme paramètres. Il retourne un vecteur contenant uniquement des chaussures de la pointure demandée.

Dans le corps de `chaussures_a_la_pointure`, nous appelons `into_iter` pour créer un itérateur qui prend possession du vecteur. Ensuite, nous appelons `filter` pour adapter cet itérateur dans un nouvel itérateur qui ne contient que les éléments pour lesquels la fermeture retourne `true`.

La fermeture capture le paramètre `pointure_chaussure` de l'environnement et compare la valeur avec la pointure de chaque chaussure, en ne gardant que les chaussures de la pointure spécifiée. Enfin, l'appel à `collect` retourne un vecteur qui regroupe les valeurs renvoyées par l'itérateur.

Le test confirme que lorsque nous appelons `chaussures_a_la_pointure`, nous

n'obtenons que des chaussures qui ont la même pointure que la valeur que nous avons demandée.

Créer nos propres itérateurs avec le trait `Iterator`

Nous avons vu que nous pouvons créer un itérateur en appelant `iter`, `into_iter` ou `iter_mut` sur un vecteur. Nous pouvons créer des itérateurs à partir d'autres types de collections de la bibliothèque standard, comme les tables de hachage. Nous pouvons aussi créer des itérateurs qui font tout ce que nous voulons en implémentant le trait `Iterator` sur nos propres types. Comme nous l'avons mentionné précédemment, la seule méthode pour laquelle nous devons fournir une définition est la méthode `next`. Une fois que nous avons fait cela, nous pouvons utiliser toutes les autres méthodes qui ont des implémentations par défaut fournies par le trait `Iterator` !

Pour preuve, créons un itérateur qui ne comptera que de 1 à 5. D'abord, nous allons créer une structure contenant quelques valeurs. Ensuite nous transformerons cette structure en itérateur en implémentant le trait `Iterator` et nous utiliserons les valeurs de cette implémentation.

L'encart 13-20 montre la définition de la structure `Compteur` et une fonction associée `new` pour créer des instances de `Compteur` :

Fichier : `src/lib.rs`

Encart 13-20 : définition de la structure `Compteur` et d'une fonction `new` qui crée des instances de `Compteur` avec une valeur initiale de 0 pour le champ `compteur`.

La structure `Compteur` a un champ `compteur`

avec une valeur de 0 pour le champ `compteur`.

Ensuite, nous allons implémenter le trait `Iterator` sur notre type `Compteur` en définissant le corps de la méthode `next` pour préciser ce que nous voulons qu'il se passe quand cet itérateur est utilisé, comme dans l'encart 13-21 :

Fichier : `src/lib.rs`

Encart 13-21 : implémentation du trait `Iterator` sur notre structure `Compteur`

Nous avons défini le type associé `Item` pour notre itérateur à `u32`, ce qui signifie que l'itérateur renverra des valeurs `u32`. Encore une fois, ne vous préoccupez pas des types associés, nous les aborderons au chapitre 19.

Nous voulons que notre itérateur ajoute 1 à l'état courant, donc nous avons initialisé `compteur` à 0 pour qu'il retourne 1 lors du premier appel à `next`. Si la valeur de

```
#[test]
fn appel_direct_a_next() {
    let mut compteur = Compteur::new();

    assert_eq!(compteur.next(), Some(1));
    assert_eq!(compteur.next(), Some(2));
    assert_eq!(compteur.next(), Some(3));
    assert_eq!(compteur.next(), Some(4));
    assert_eq!(compteur.next(), Some(5));
    assert_eq!(compteur.next(), None);
}
```

Encart 13-22 : test de l'implémentation de la méthode `next`

Ce test crée une nouvelle instance de `Compteur` dans la variable `compteur` et appelle ensuite `next` à plusieurs reprises, en vérifiant que nous avons implémenté le comportement que nous voulions que cet itérateur suive : renvoyer les valeurs de 1 à 5.

Utiliser d'autres méthodes du trait `Iterator`

Maintenant que nous avons implémenté le trait `Iterator` en définissant la méthode `next`, nous pouvons maintenant utiliser les implémentations par défaut de n'importe quelle méthode du trait `Iterator` telles que définies dans la bibliothèque standard, car elles utilisent toutes la méthode `next`.

Par exemple, si pour une raison quelconque nous voulions prendre les valeurs produites par une instance de `Compteur`, les coupler avec des valeurs produites par une autre instance de `Compteur` après avoir sauté la première valeur, multiplier chaque paire ensemble, ne garder que les résultats qui sont divisibles par 3 et additionner toutes les valeurs résultantes ensemble, nous pourrions le faire, comme le montre le test dans l'encart 13-23 :

Fichier : `src/lib.rs`

```
#[test]
fn utilisation_des_autres_methodes_du_trait_iterator() {
    let somme: u32 = Compteur::new()
        .zip(Compteur::new().skip(1))
        .map(|(a, b)| a * b)
        .filter(|x| x % 3 == 0)
        .sum();
    assert_eq!(18, somme);
}
```

Encart 13-23 : utilisation d'une gamme de méthodes du trait `Iterator` sur notre itérateur `Compteur`

Notez que `zip` ne produit que quatre paires ; la cinquième paire théorique `(5, None)` n'est jamais produite car `zip` retourne `None` lorsque l'un de ses itérateurs d'entrée retourne `None`.

Tous ces appels de méthode sont possibles car nous avons renseigné comment la méthode `next` fonctionne et la bibliothèque standard fournit des implémentations par défaut pour les autres méthodes qui appellent `next`.

Amélioration de notre projet d'entrée/sortie

Grâce à ces nouvelles connaissances sur les itérateurs, nous pouvons améliorer le projet d'entrée/sortie du chapitre 12 en utilisant des itérateurs pour rendre certains endroits du code plus clairs et plus concis. Voyons comment les itérateurs peuvent améliorer notre implémentation de la fonction `Config::new` et de la fonction `rechercher`.

Supprimer l'appel à `clone` à l'aide d'un itérateur

Dans l'encart 12-6, nous avons ajouté du code qui prenait une *slice* de `String` et qui créait une instance de la structure `Config` en utilisant les indices de la *slice* et en clonant les valeurs, permettant ainsi à la structure `Config` de posséder ces valeurs. Dans l'encart 13-24, nous avons reproduit l'implémentation de la fonction `Config::new` telle qu'elle était dans l'encart 12-23 à la fin du chapitre 12 :

Fichier : `src/lib.rs`

```
impl Config {
    pub fn new(args: &[String]) -> Result<Config, &'static str> {
        if args.len() < 3 {
            return Err("il n'y a pas assez d'arguments");
        }

        let recherche = args[1].clone();
        let nom_fichier = args[2].clone();

        let sensible_casse = env::var("MINIGREP_INSENSIBLE_CASSE").is_err();

        Ok(Config {
            recherche,
            nom_fichier,
            sensible_casse,
        })
    }
}
```

Encart 13-24 : reproduction de la fonction `Config::new` de la fin du chapitre 12

À ce moment-là, nous avons dit de ne pas s'inquiéter des appels inefficaces à `clone` parce que nous les supprimerions à l'avenir. Et bien, ce moment est venu !

Nous avons besoin de `clone` ici parce que nous avons une slice d'éléments `String`

dans le paramètre `args`, mais la fonction `new` ne possède pas `args`. Pour renvoyer la propriété d'une instance de `Config`, nous avons dû cloner les valeurs des champs `recherche` et `nom_fichier`

Fichier : src/lib.rs

```
impl Config {
    pub fn new(mut args: env::Args) -> Result<Config, &'static str> {
        args.next();

        let recherche = match args.next() {
            Some(arg) => arg,
            None => return Err("nous n'avons pas de chaîne de caractères"),
        };

        let nom_fichier = match args.next() {
            Some(arg) => arg,
            None => return Err("nous n'avons pas de nom de fichier"),
        };

        let sensible_casse = env::var("MINIGREP_INSENSIBLE_CASSE").is_err();

        Ok(Config {
            recherche,
            nom_fichier,
            sensible_casse,
        })
    }
}
```

Encart 13-27 : changement du corps de `Config::new` afin d'utiliser les méthodes d'itération

Rappelez-vous que la première valeur de ce qui est retourné par `env::args` est le nom du programme. Nous voulons ignorer cette valeur et passer à la suivante, donc d'abord nous appelons une fois `next` et nous ne faisons rien avec sa valeur de retour. Ensuite, nous appelons `next` pour obtenir la valeur que nous voulons mettre dans le champ `recherche` de `Config`. Si `next` renvoie un `Some`, nous utilisons un `match` pour extraire sa valeur. S'il retourne `None`, cela signifie que pas assez d'arguments ont été fournis, si bien que nous quittons aussitôt la fonction en retournant une valeur `Err`. Nous procédons de même pour la valeur `nom_fichier`.

Rendre le code plus clair avec des adaptateurs d'itération

Nous pouvons également tirer parti des itérateurs dans la fonction `rechercher` de notre projet d'entrée/sortie, qui est reproduite ici dans l'encart 13-28, telles qu'elle était dans l'encart 12-19 à la fin du chapitre 12 :

Fichier : src/lib.rs

```
pub fn rechercher<'a>(recherche: &'a str, contenu: &'a str) -> Vec<'a str> {
    let mut resultats = Vec::new();

    for ligne in contenu.lines() {
        if ligne.contains(recherche) {
            resultats.push(ligne);
        }
    }

    resultats
}
```

Encart 13-28 : La mise en oeuvre de la fonction `rechercher` de l'encart 12-19

Nous pouvons écrire ce code de façon plus concise en utilisant des méthodes des adaptateurs d'itération. Ce faisant, nous évitons ainsi d'avoir le vecteur mutable `resultats`. Le style de programmation fonctionnelle préfère minimiser la quantité d'états modifiables pour rendre le code plus clair. Supprimer l'état mutable pourrait nous aider à faire une amélioration future afin que la recherche se fasse en parallèle, car nous n'aurions pas à gérer l'accès concurrent au vecteur `resultats`. L'encart 13-29 montre ce changement :

Fichier : src/lib.rs

```
pub fn rechercher<'a>(recherche: &'a str, contenu: &'a str) -> Vec<'a str> {
    contenu
        .lines()
        .filter(|ligne| ligne.contains(recherche))
        .collect()
}
```

Encart 13-29 : utilisation des méthodes des adaptateurs d'itération dans l'implémentation de la fonction `rechercher`

Souvenez-vous que le but de la fonction `rechercher` est de renvoyer toutes les lignes dans `contenu` qui contiennent `recherche`. Comme dans l'exemple de `filter` dans l'encart 13-19, nous pouvons utiliser l'adaptateur `filter` pour garder uniquement les lignes pour lesquelles `ligne.contains(recherche)` renvoie `true`. Nous collectons ensuite les lignes correspondantes dans un autre vecteur avec `collect`. C'est bien plus simple ! N'hésitez pas à faire le même changement pour utiliser les méthodes d'itération dans la fonction `rechercher_insensible_casse`.

Logiquement la question suivante est de savoir quel style utiliser dans votre propre code et pourquoi : l'implémentation originale de l'encart 13-28 ou la version utilisant l'itérateur dans l'encart 13-29. La plupart des développeurs Rust préfèrent utiliser le style avec l'itérateur. C'est un peu plus difficile à comprendre au début, mais une fois que vous avez compris les différents adaptateurs d'itération et ce qu'ils font, les itérateurs peuvent devenir plus faciles à comprendre. Au lieu de jongler avec différentes boucles et de construire de nouveaux vecteurs, ce code se concentre sur l'objectif de haut niveau de la boucle. Cette abstraction permet d'éliminer une partie du code trivial, de sorte qu'il soit plus facile de dégager les concepts propres à ce code, comme le filtrage de chaque élément de l'itérateur qui est appliqué.

Mais ces deux implémentations sont-elles réellement équivalentes ? L'hypothèse intuitive pourrait être que la boucle de plus bas niveau sera plus rapide. Intéressons nous donc maintenant à leurs performances.